

AI System Security: Building, Breaking, and Hardening LLM/Agentic Systems

This is the lifecycle view: how to *build* an LLM/agent system that is defensible, how to *break* it (red team) to find the gaps, and how to *harden* it so the gaps close and stay closed in production.

Build: secure-by-construction

The most important architectural decision is **where authority lives**. In a secure design, the LLM is an untrusted planner that emits *proposals*; a deterministic control plane validates and executes them under the caller's permissions. Practical build rules:

- **Two planes.** Separate the reasoning plane (model) from the action plane (tools/effects). All authority checks happen in the action plane.
- **Schema everything.** Tool inputs and outputs have schemas; validate both.
- **Default deny.** Tools, hosts, file paths, and data scopes are allow-listed.
- **Stateless secrets.** Secrets are fetched at execution time from a vault, never embedded in prompts or code (`airte.cloud.secrets`).
- **Observability from day one.** Every model call, tool call, and guardrail decision is logged with a correlation ID. You cannot defend what you cannot see.

Break: red-teaming the system

Breaking is structured, repeatable, and measured — not ad-hoc poking. Use a harness that runs attack suites and scores an **attack-success-rate (ASR)**.

Live targets ship for Anthropic, OpenAI, Bedrock, Azure OpenAI, Vertex, Mistral, Cohere, and Ollama (`airte.redteam.build_target`), each guarded with input injection heuristics and output redaction.

```
from airte.redteam import RedTeamHarness

class MySystem:
    def generate(self, prompt: str) -> str:
        return my_agent.run(prompt)

report = RedTeamHarness(MySystem()).run("all")
print(report.summary())           # ASR + which attacks succeeded
```

What to throw at it:

- **Injection (direct & indirect).** Plant payloads in user input *and* in any document/tool output the agent will read. Indirect is where real systems fall.

- **Jailbreaks.** Persona, hypothetical, refusal-suppression, payload-splitting.
- **Poisoning.** Insert a malicious doc into the RAG index and confirm it cannot steer answers or actions.
- **Extraction.** Try to recover the system prompt and any injected secrets.
- **Excessive agency probes.** Ask the agent to take an action outside its scope and confirm the guardrail — not the model's good judgment — blocks it.
- **Tool-chain abuse.** Combine tools (read file -> exfiltrate via fetch) to test whether the *composition* is safe, not just each tool individually.

The `examples/vulnerable_mcp_server.py` + scanner, and the EchoTarget baseline, let you validate the harness itself before pointing it at a real system.

Harden: closing and keeping gaps closed

Hardening converts findings into controls and controls into regression tests:

1. **Input guardrails** — normalize (defang homoglyphs/zero-width), heuristically flag injections, and *mark* rather than silently drop suspicious input so the model treats it as data (`airte.guardrails.input_filters`).
2. **Retrieval guardrails** — fence and quarantine retrieved content (`airte.guardrails.rag_guardrails`).
3. **Agent guardrails** — scoped permissions, step/cost budgets (loop guard), per-tool rate limits, and human approval for irreversible actions (`airte.guardrails.agent_guardrails`).
4. **Output guardrails** — schema-validate, encode for the sink, and redact secrets/PII (`airte.guardrails.output_filters`, `airte.data_pipeline.dlp`).
5. **Platform hardening** — sandbox tool execution, deny egress by default, run each component under a least-privilege identity (see `cloud-security-ai.md`).
6. **Regression gates** — every fixed finding becomes a red-team case in CI with an ASR threshold. A guardrail regression now fails the build.

Production operations

- **Detection.** Alert on guardrail trips, authz denials, step-budget exhaustion, anomalous tool-call rates, and DLP hits.
- **Rate & cost controls.** Enforce token/size/call/spend caps to prevent unbounded consumption (LLM10) and denial-of-wallet.
- **Incident readiness.** Pre-author a kill switch: feature-flag tools off, rotate the model API key, and revoke the agent's service identity. Treat a manipulated agent like a compromised service account.
- **Change management.** Model/prompt/tool changes go through review and re-run the red-team suite, because a prompt edit can silently reopen a closed vulnerability.